



**YOUR TECHNOLOGY & INNOVATION PARTNER**

**Advanced AI Training - Session 3**

# **Advanced Natural Language Processing (NLP)**

# Table of Contents

1.

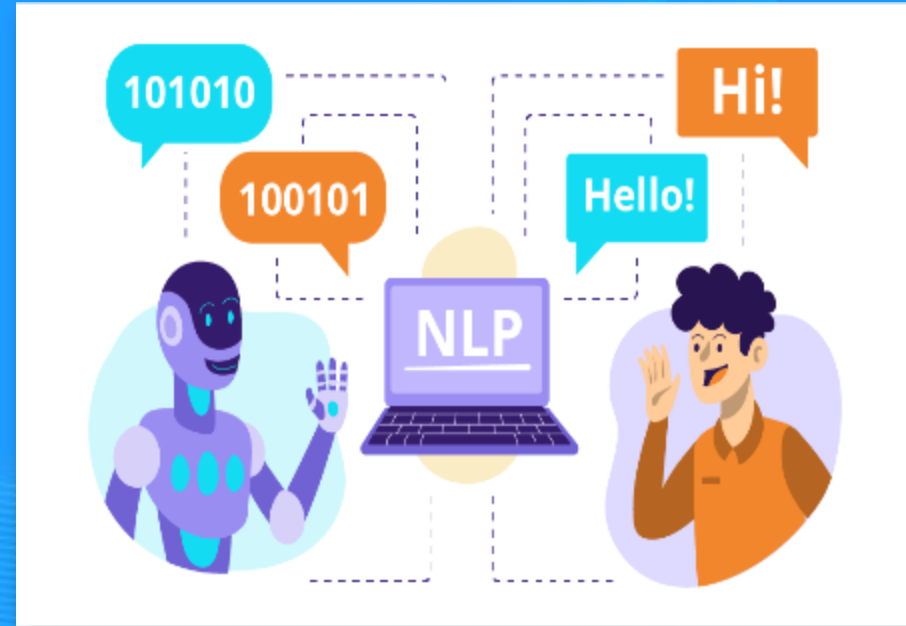
Evolution of NLP

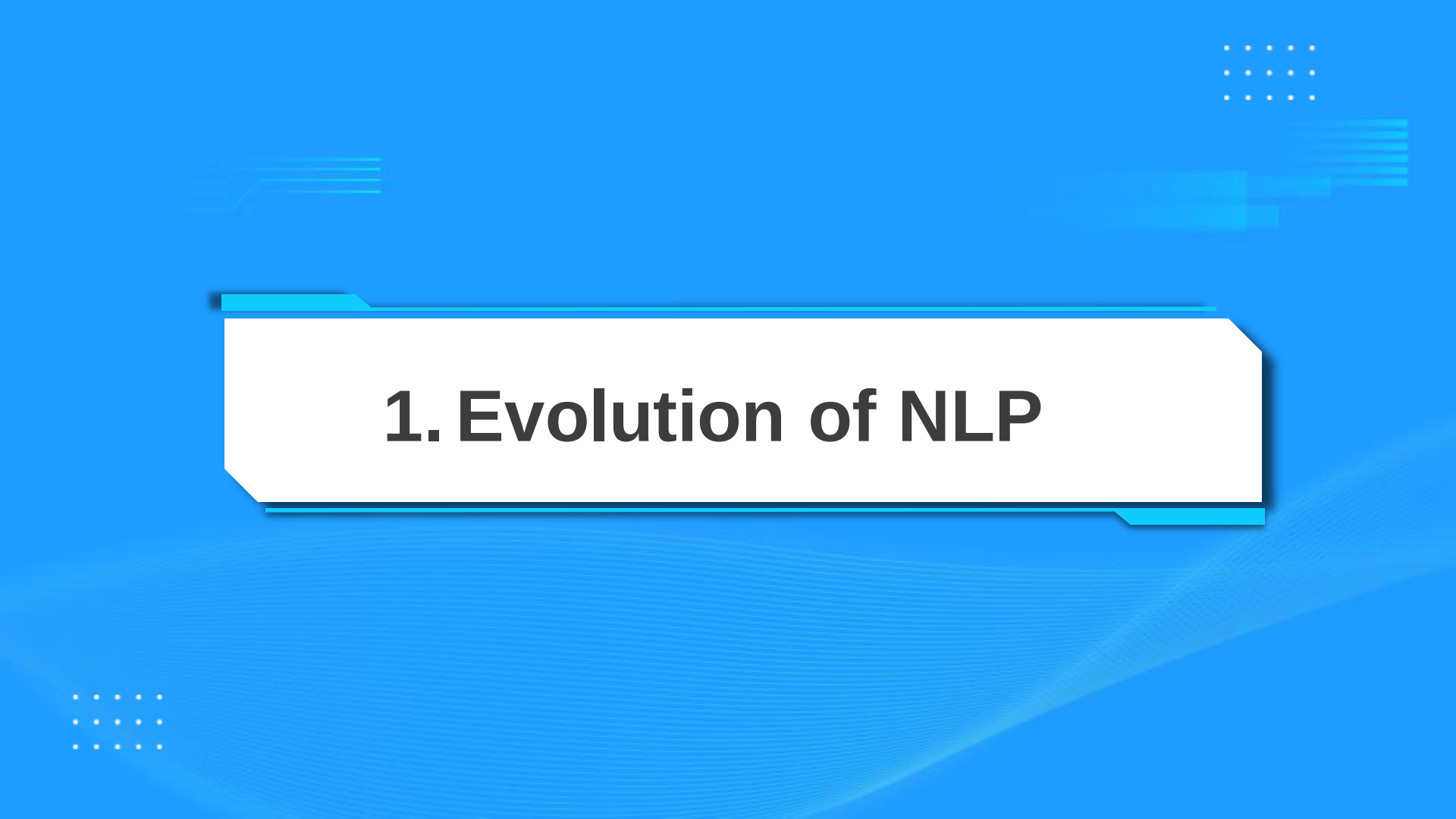
2.

AI Agents

3.

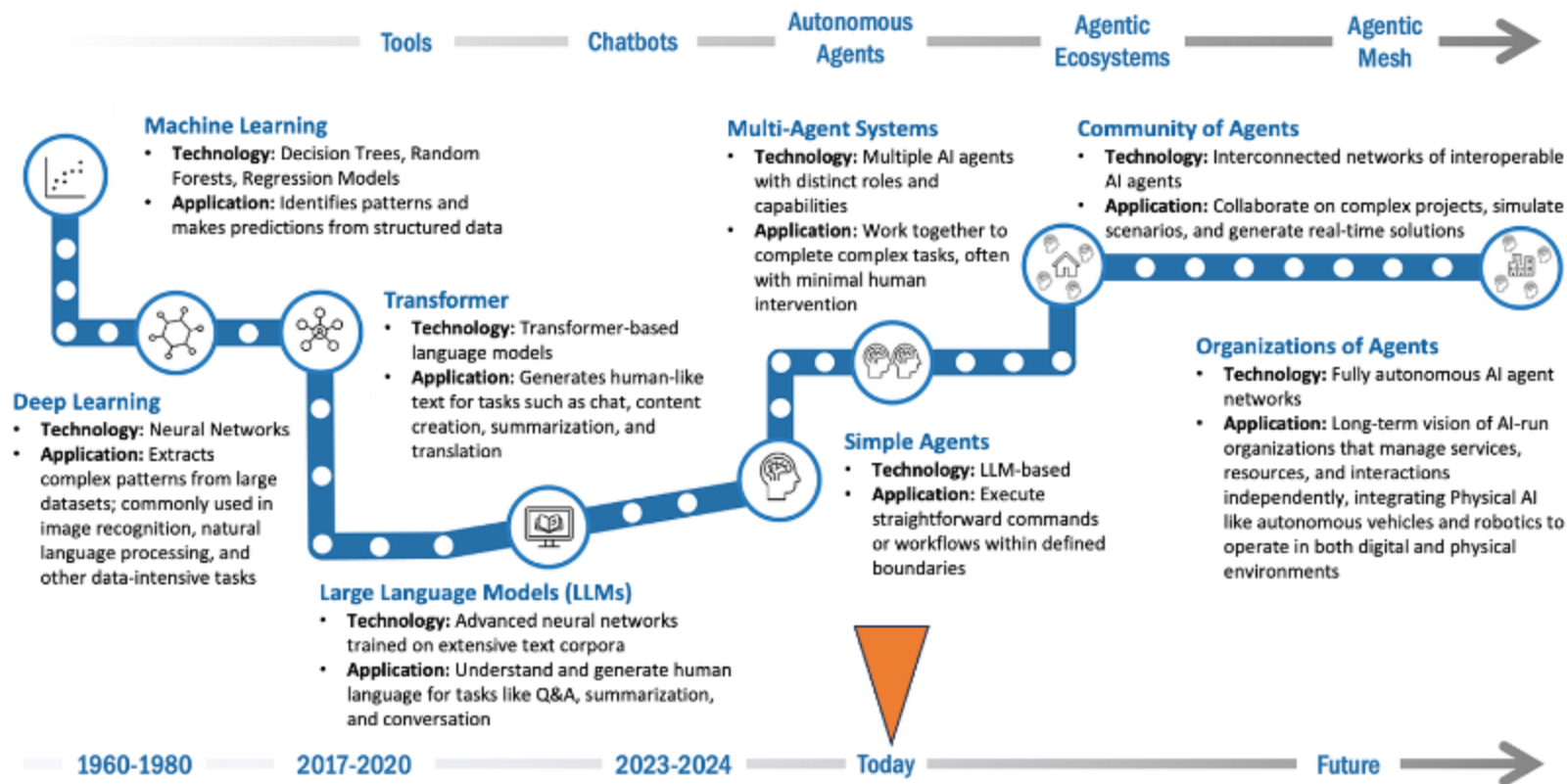
Demo of some Tools





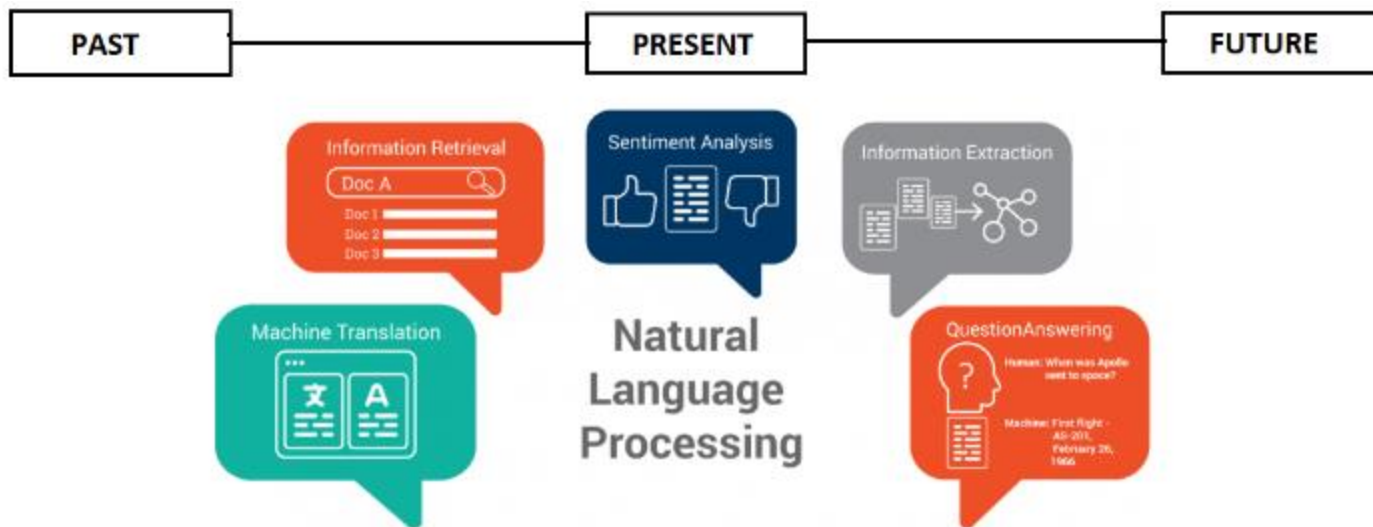
# **1. Evolution of NLP**

# Evolution of Language Models



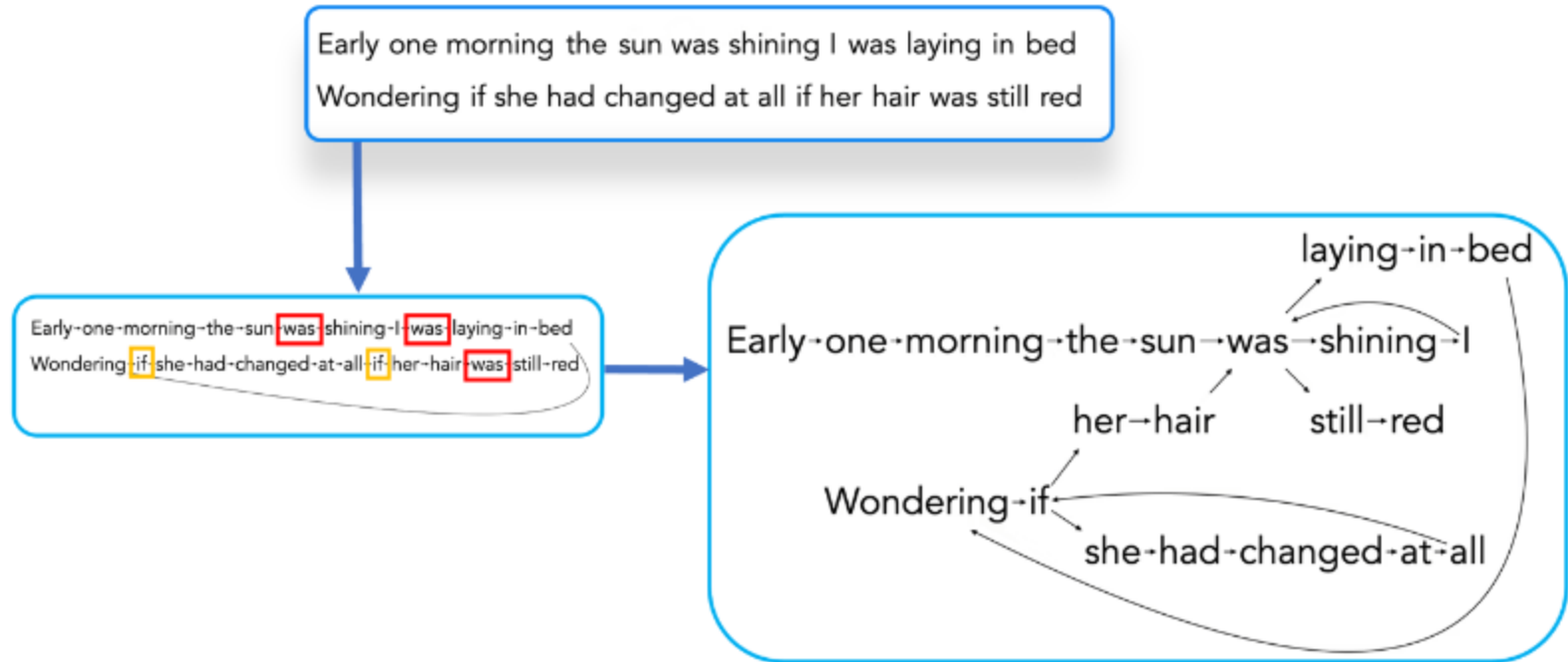
# Covered NLP Models

- Recurrent Neural Network (RNNs)
- Long-Short Terms Memory Networks (LSTM)
- Transformers



# Early Age of Language Modelling

By tracing repeated words, we can build a tree that generates new sentences.

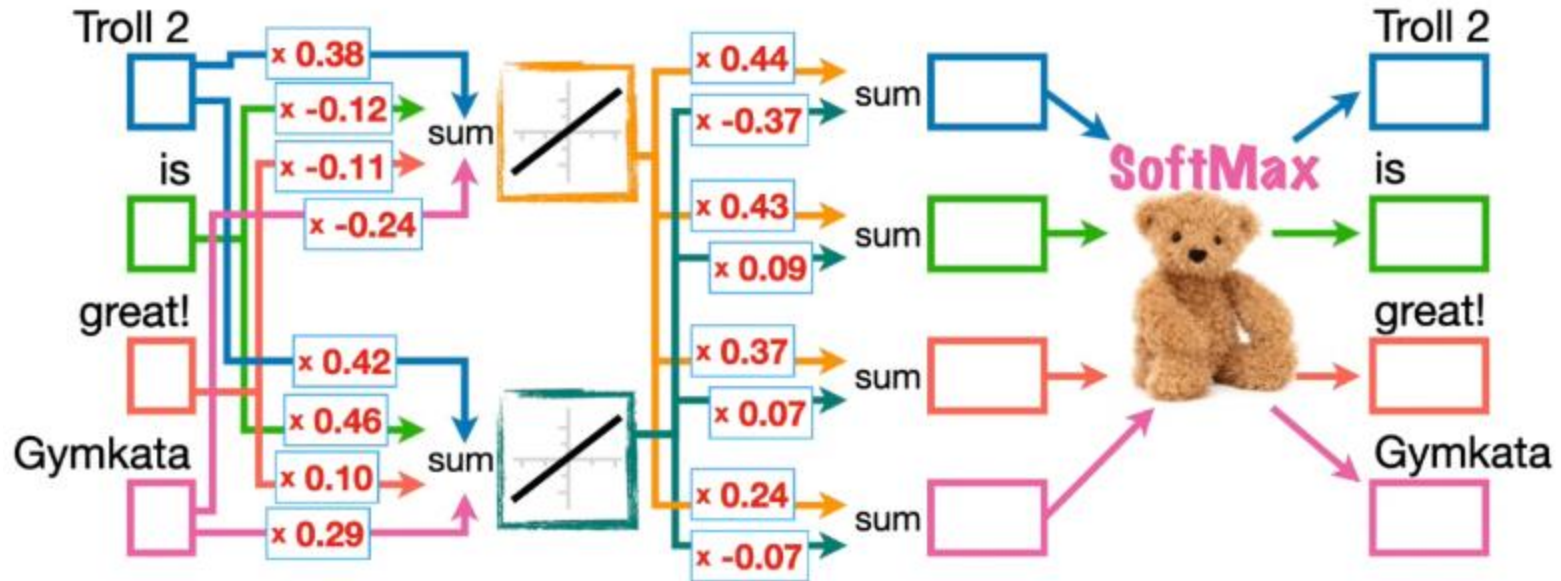


# Word Embedding: Word2Vec (1/2)

## Training Data

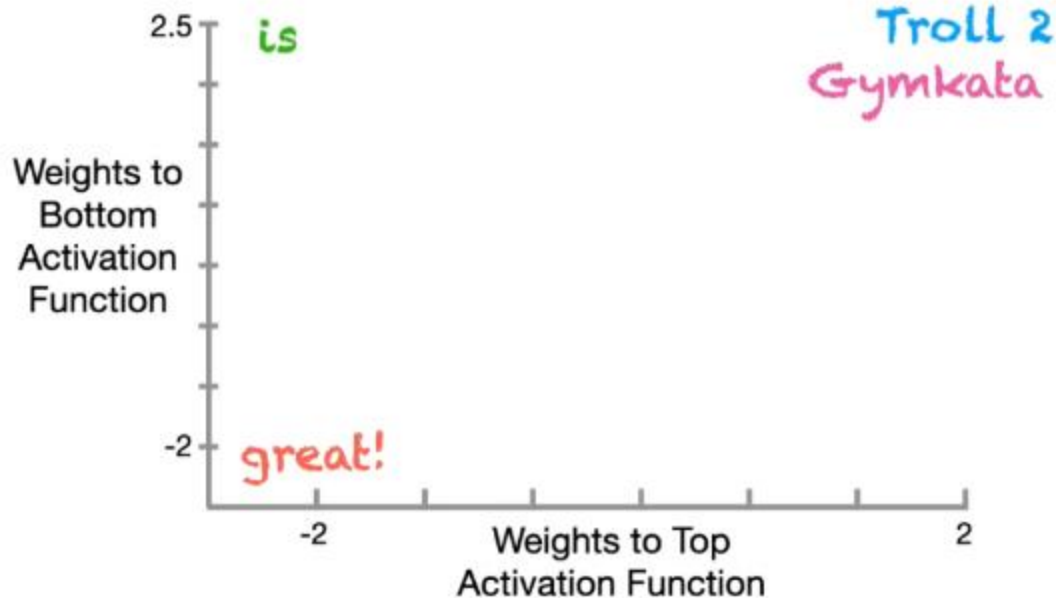
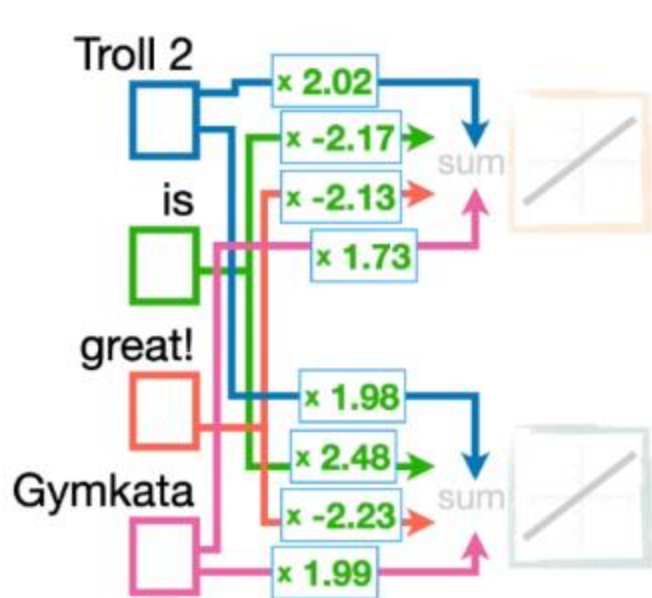
- Troll 2 is great
- Gymkata is great

Word2Vec is a model that creates word embeddings using a neural network. It learns word meanings by predicting surrounding words.



## Word Embedding & Word2Vec (2/2)

After the neural network feedforward and backpropagation steps, similar words end up with vector representations that are closer together in the embedding space.





# Recurrent Neural Networks (RNNs)

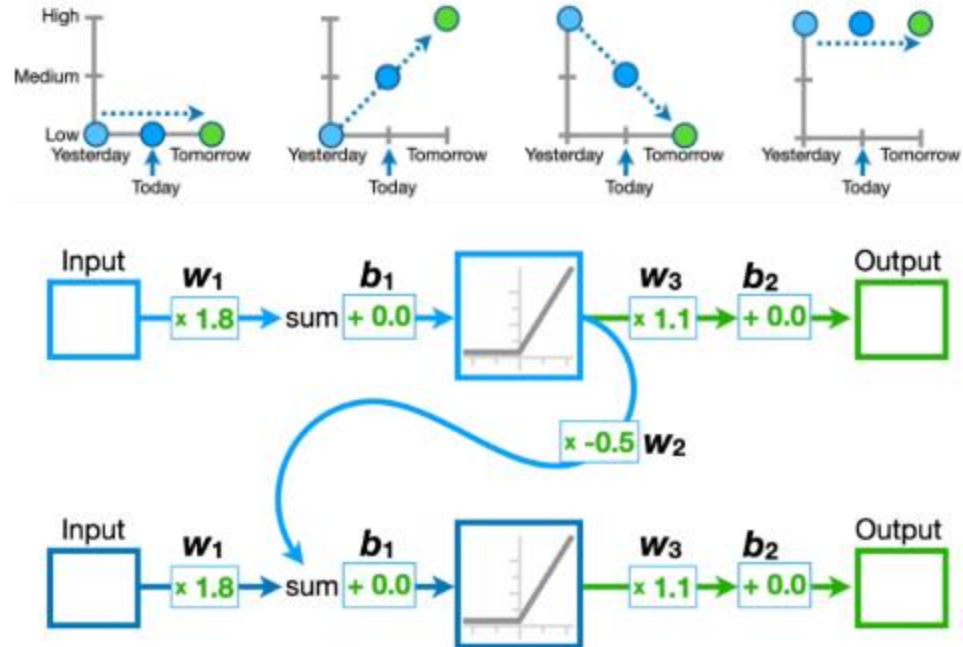
RNNs were introduced as neural networks designed to work with sequential data of varying lengths

## Step-by-Step Processing:

1. At each time step, the RNN takes an input.
2. It combines the current input with its previous memory (hidden state) to compute a new hidden state.
3. The updated hidden state is used for predictions or decisions.

## Example: Word Prediction

- “I” → Updates memory.
- “love” → Updates with “I” and “love.”
- “programming” → Updates with “I,” “love,” and “programming.”

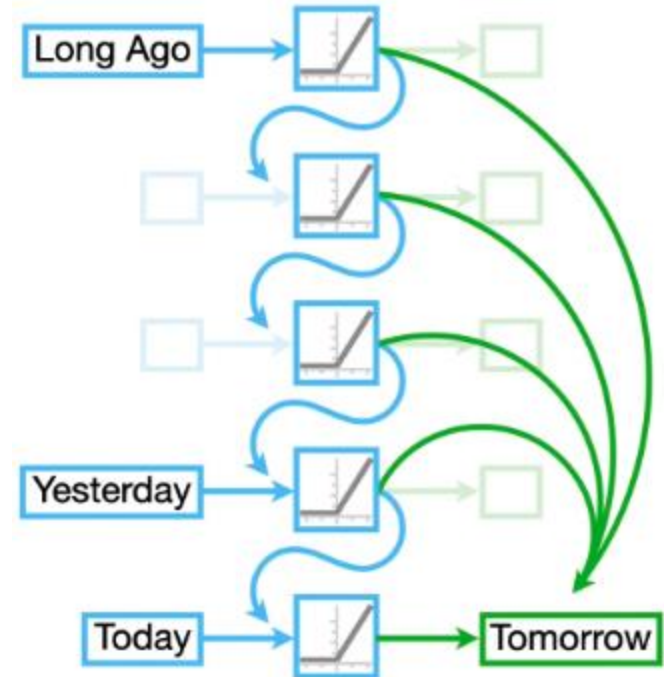


# Long-Short Terms Memory Networks (LSTMs)

LSTMs were introduced as a type of neural network specifically designed to overcome the vanishing and exploding gradient problems commonly encountered during the training of RNNs.

## Step-by-Step Processing:

1. At each time step, the LSTM takes an input.
2. It checks what information from the previous memory should be forgotten (forget gate).
3. It decides what new information should be remembered (input gate).
4. It updates the internal memory with a mix of old and new information (cell state).
5. It decides what to output (output gate), which becomes the hidden state for the next step.
6. The updated hidden state is used for predictions or decisions.



# Transformer in Neural Network

How Neural Network with Attention mechanism used to predict next word

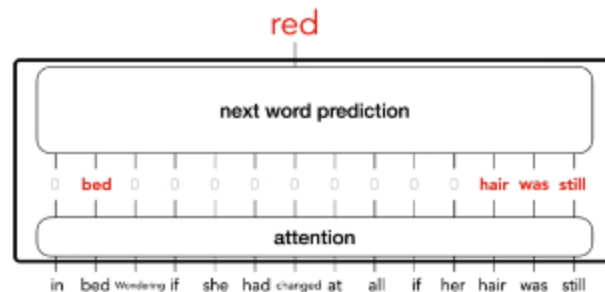
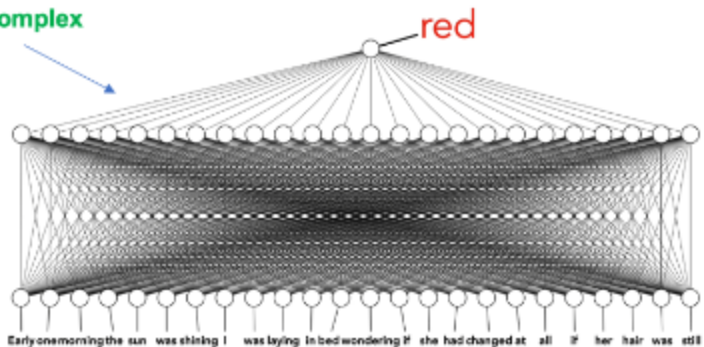
Predict this word

Early one morning the sun was shining I was laying in bed  
Wondering if she had changed at all if her hair was still red

Still predictable

\_\_\_\_\_ bed  
\_\_\_\_\_ hair was still red

Too complex



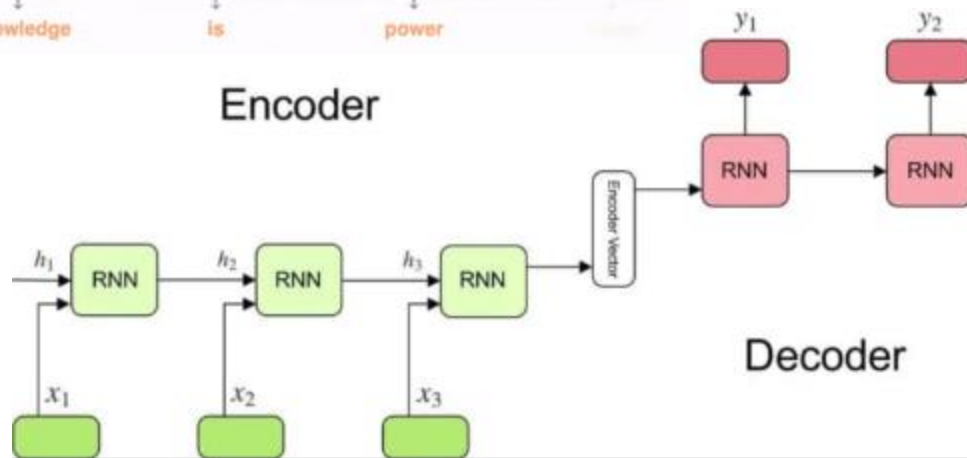
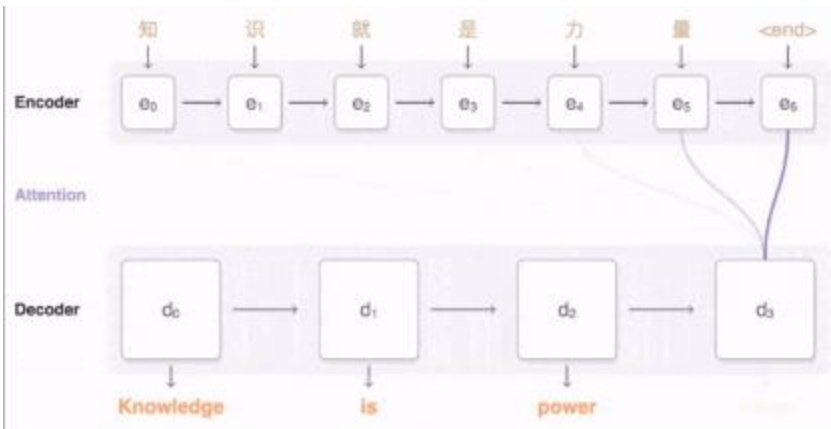
Word Tokenize

She is beautiful ➡ She / is / beau / ti / ful

# Seq2Seq: Encoder-Decoder Neural Networks

The **Seq2Seq** model consists of two main components: the **encoder** and the **decoder**, often implemented with RNNs or LSTMs.

- The **encoder** processes the input sequence (e.g., a sentence) and compresses it into a fixed-length vector, known as the **context vector**. This vector contains all the relevant information from the input.
- The **decoder** then uses this context vector to generate the output sequence (e.g., a translated sentence), one token at a time.



# Types of Transformer

## Encoder-only

**Example:** BERT, RoBERTa, DistilBERT

**Use cases:** Sentiment analysis, question answering, NER

## Decoder-only

**Example:** GPT, Grok, Phi, Mistral

**Use cases:** Text generation, code generation, auto-complete, chatbots.

## Encoder-Decoder

**Example:** T5, mT5, BART

**Use cases:** Translation, summarization, text-to-text generation.

## Vision Transformer

**Example:** ViT

**Use cases:** Image classification, object detection, segmentation.

## Multi-modal Transformer

**Example:** GPT-4V, CLIP

**Use cases:** Image captioning, VQA, document processing.



## 2. AI Agents

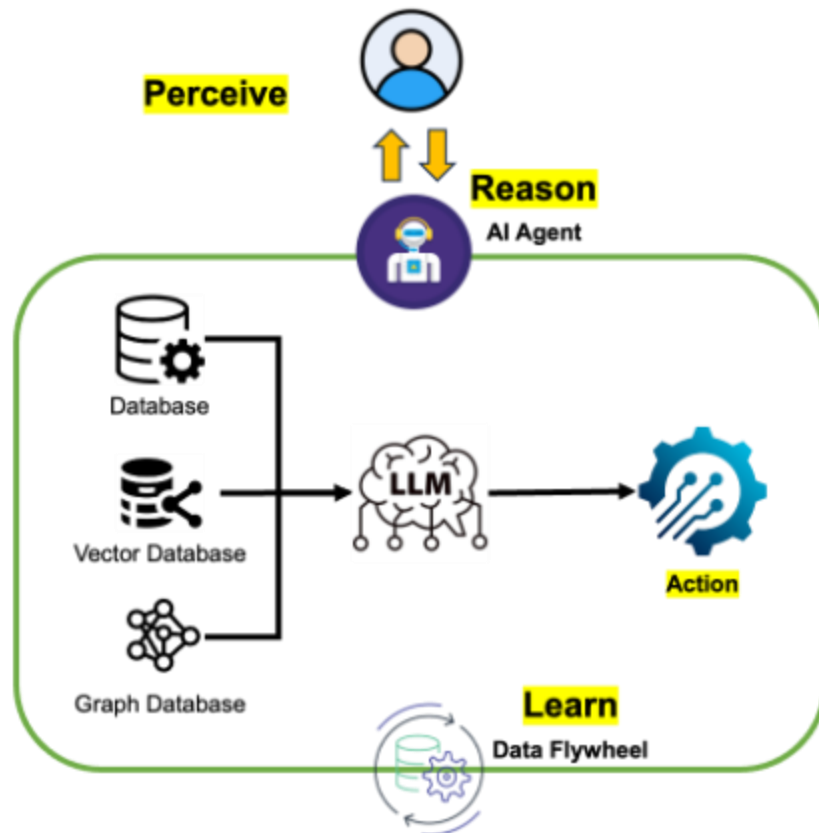
# What is AI Agents?

**Perceive:** AI agent gathers and analyzes data from sources like sensors, databases, and interfaces, identifying key features and relevant entities.

**Reason:** A language model coordinates tasks, generates solutions, and directs specialized tools, using methods like RAG to pull accurate, context-aware insights from proprietary data.

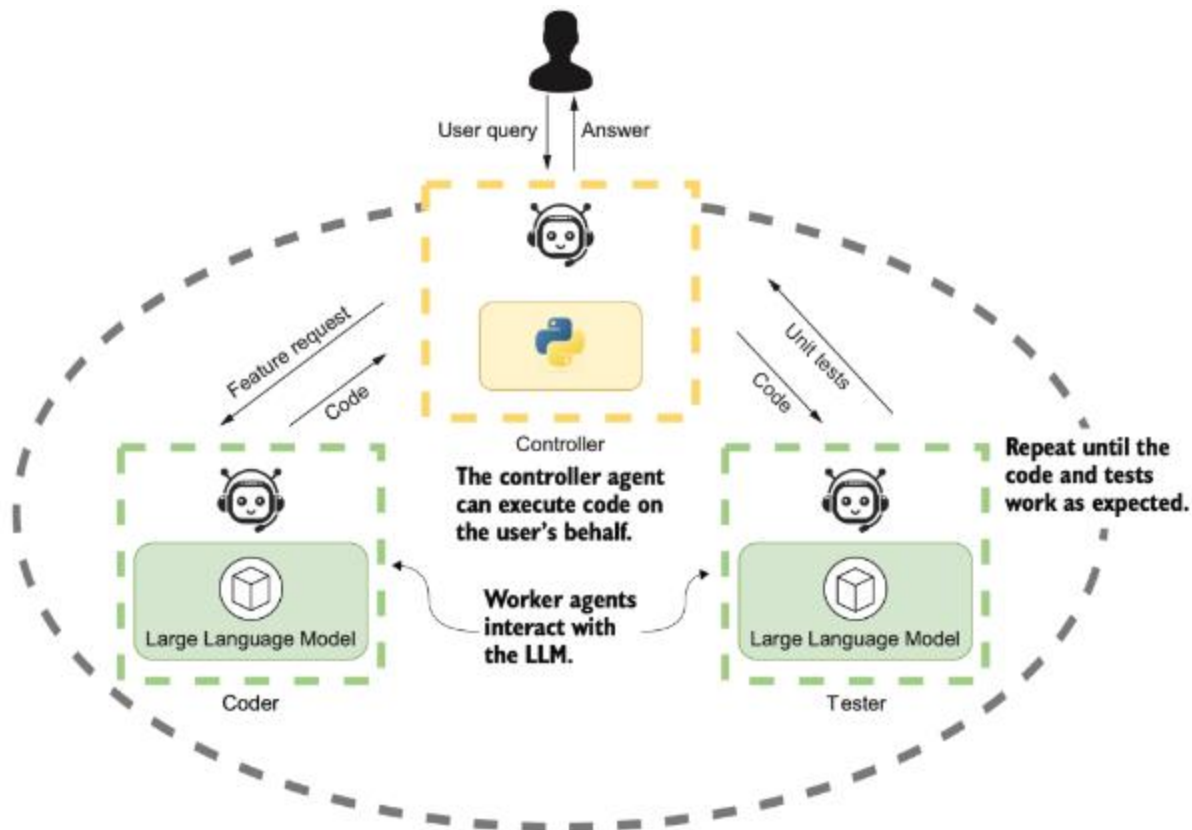
**Act:** It executes plans through API integrations, with built-in guardrails to ensure correct actions; for instance, automating claims under a limit while flagging higher ones for human review.

**Learn:** Through continuous feedback, the system refines its performance, becoming more accurate and efficient over time.



# AI Agent Example

This is an example of a multi-agent system, the controller or agent proxy communicate directly with the user. Two agents, a coder and a tester, work in the background to create code and write unit tests to test the code.

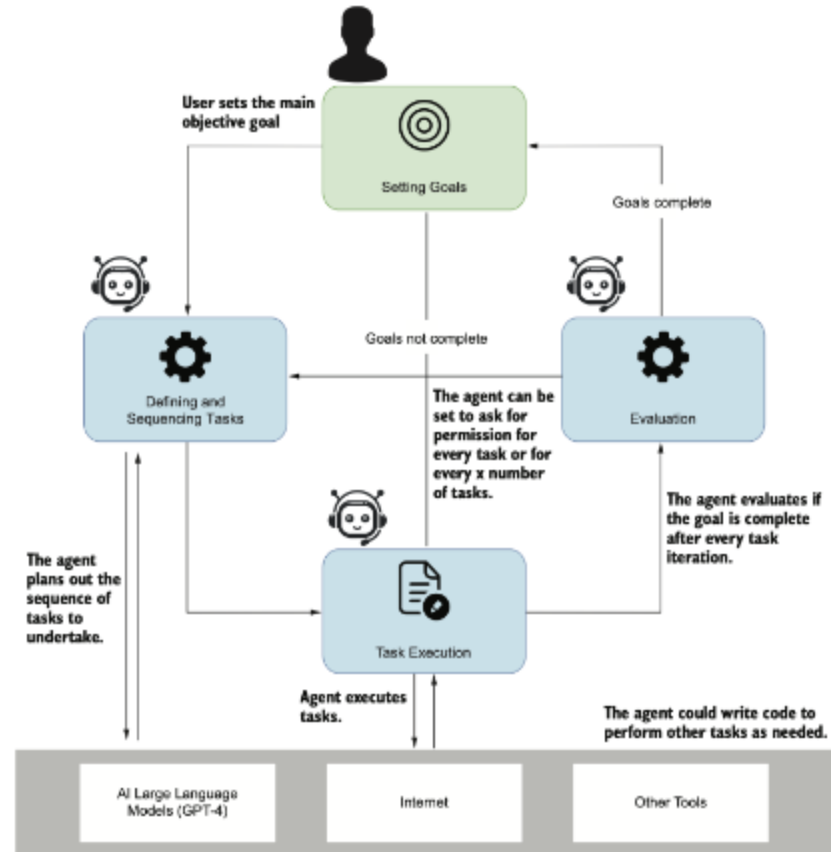




# Original Design of AutoGPT agent System

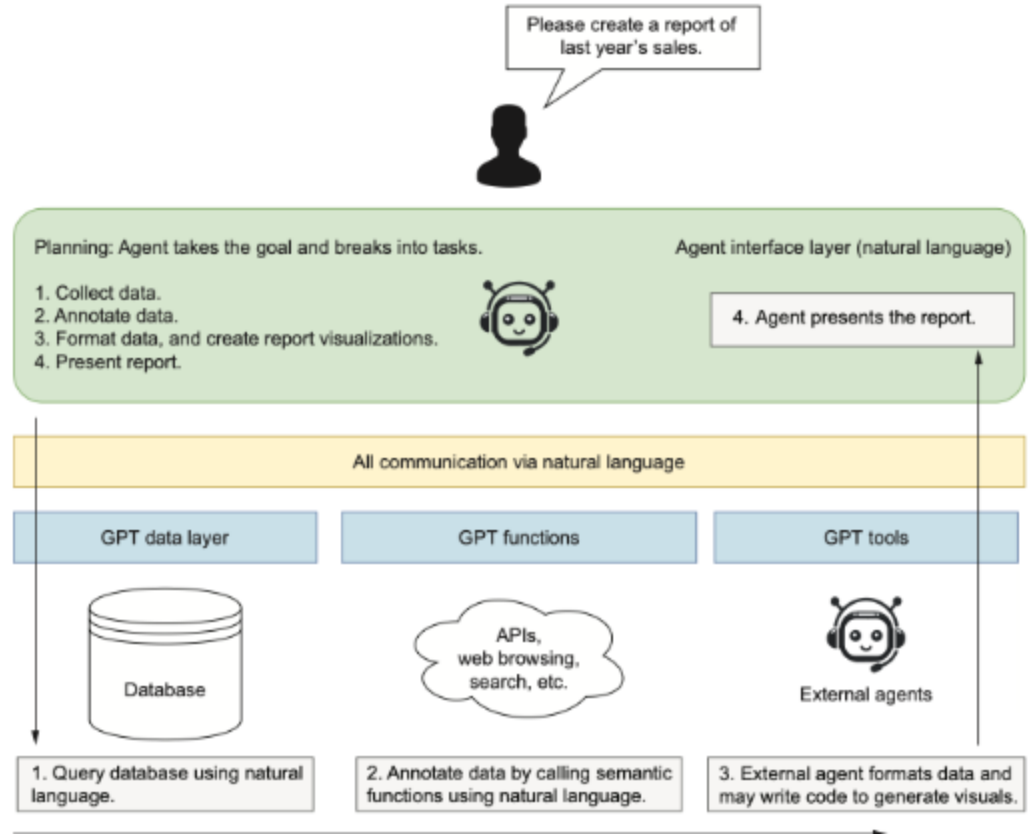
## Autonomous AI Agent

Effective prompting is an iterative process that requires continuous reflection and refinement. **AutoGPT**, one of the first autonomous agent systems, embodies this by analyzing a user's goal and generating a sequence of tasks to achieve it. As it executes each task, the agent evaluates progress toward the goal and determines whether adjustments are needed. If the goal hasn't been met, it updates its plan, revises the steps, and incorporates new information or human feedback to improve its approach.



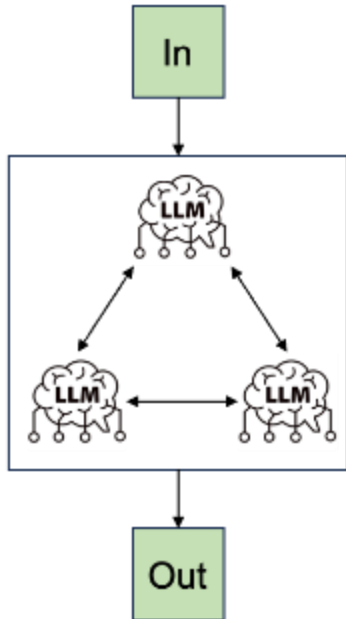
# Vision of AI Agents

The AI agent paradigm is not only a shift in how we work with LLMs but is also perceived as a shift in how we develop software and handle data. Software and data will no longer be interfaced using user interfaces (UIs), application programming interfaces (APIs), and specialized query languages such as SQL. **Instead, they will be designed to be interfaced using natural language.**



# Multi-Agents Approaches

There are two common approaches for designing multi-agent system.



## Network (or Horizontal)

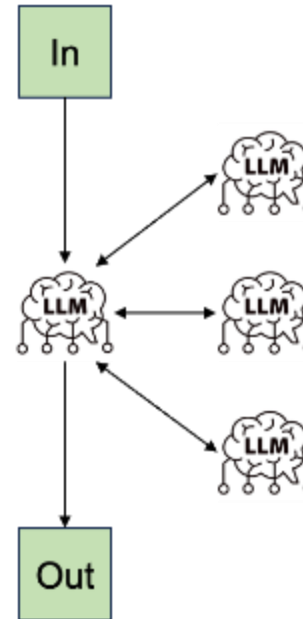
Agents communicate directly with one another in a many-to-many fashion forming a decentralized network.

### Pros:

- Distributed collaboration and group-driven decision-making.
- The system remains functional even if some agents fail.

### Cons:

- Managing communication among agents can become challenging.
- More communication may cause inefficiencies and the possibility of agents duplicating efforts.



## Hierarchical (or Vertical)

Agents are organized in a tree-like structure, with higher-level agents (supervisor agents) managing lower-level ones.

### Pros:

- Clear division of roles and responsibilities among agents at different levels.
- Streamlined communication
- Suitable for large systems with a structured decision flow.

### Cons:

- Failure at upper levels can disrupt the entire system.
- Lower-level agents have limited independence.

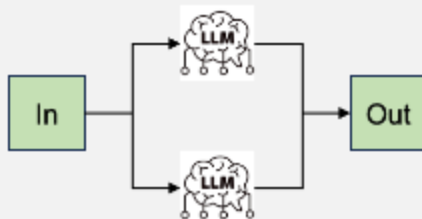
# Interaction Paradigms of Multi-Agents

There are four key interaction paradigms for orchestrating multiple agents working together.

## Parallel

Multiple AI agents run at the same time, each focusing on a different part or aspect of a larger task.

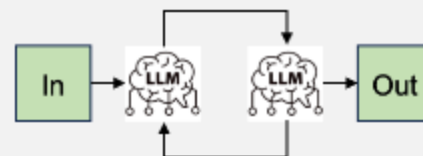
**Benefits:** Speed up processing by dividing and conquering



## Loop

Agents collaborate in cycles, revising outputs based on feedback from each other. This supports iterative refinement.

**Benefits:** Continuous improvement and better final results.



## Sequential

Agents work one after another, like steps in a pipeline. The output of one is the input to the next.

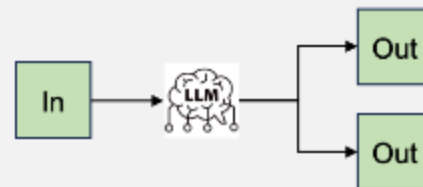
**Benefits:** Ensures structured flow and dependent logic is respected.



## Router

A central agent or logic decides which agents to activate based on the task, user query, or context.

**Benefits:** Efficient resource usage and task-specialized delegation.



# Strategies for Writing Prompts

|          | Detailed Queries                                                                                                                                                                                                                                                                              | Adopting Personas                                                                                                                                                                                                                                    | Using Delimiters                                                                                                                                                                                            | Specifying Steps                                                                                                                                                                                                                                                                                                                                                                 | Providing Examples                                                                                                                                                                                                                                                                                                                                           | Specify Output Length                                                                                                                                                                                                        |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| EXAMPLES | <p><b>Without detailed queries:</b><br/>Who's the prime minister?</p> <p><b>With detailed queries:</b><br/>Who is the prime minister of Canada, and how frequently are elections held?</p> <p><b>Provide as much detail as you can in a query; generally, the more detail the better.</b></p> | <p><b>SYSTEM:</b><br/>You are a prompt expert and will suggest ways to improve a user request.</p> <p><b>USER:</b><br/>What is the capital of Canada?</p> <p><b>Personas can include details about demographics, knowledge, and personality.</b></p> | <p><b>USER:</b><br/>Summarize the text delimited by triple quotes with a limerick:<br/>"text to be summarized"</p> <p><b>Delimiters can help separate blocks of content from specification details.</b></p> | <p><b>SYSTEM:</b><br/>Use the following step-by-step instructions to respond to user inputs:<br/><b>Step 1</b> - Summarize the text in triple quotes to one sentence with a prefix that says "Summary: ".<br/><b>Step 2</b> - Translate the summary from Step 1 into French, with a prefix that says "Translation: ".<br/><b>USER:</b><br/>"text to summarize and translate"</p> | <p><b>SYSTEM:</b><br/>Answer in a consistent style.</p> <p><b>USER:</b><br/>Teach me about patience.</p> <p><b>This is the example.</b></p> <p><b>ASSISTANT:</b> ←<br/>The river that carves the deepest valley flows from a modest spring; the most intricate tapestry begins with a solitary thread.</p> <p><b>USER:</b><br/>Teach me about the ocean.</p> | <p><b>USER:</b><br/>Summarize the text delimited by triple quotes in about 50 words.<br/>"text to summarize here"</p> <p><b>Limiting the length of output can be specific to words, bullet points, or other metrics.</b></p> |

# AI Agents vs Agentic AI

Agentic AI refers to **AI systems** that **act autonomously** and **make decisions** on behalf of a user or organization based on specific goals, often without human intervention.

## Key Differentiators

Goal

Autonomy

Adaptation



AI Agents

Automate simple tasks



Agentic AI

Make autonomous decisions

### **3. Demo of some Tools**

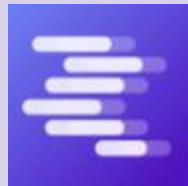
# Tools to build AI Agents



## OpenAI Agent SDK

The OpenAI Agent SDK lets developers build AI agents that can use tools, APIs, and perform tasks autonomously within custom workflows.

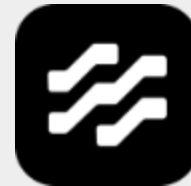
Source: [Link](#)



## LMStudio

LM Studio is a free desktop app that lets you run open-source large language models locally, with a simple interface for chatting, exploring models, and serving APIs offline.

Source: [Link](#)



## Langflow

Langflow is an open-source, low-code tool for visually designing and deploying AI workflows and agents through a drag-and-drop interface.

Source: [Link](#)





**THANK YOU!**